

Week 6 Practical

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
divya@Divyasree:~$ mkdir fifo_project
divya@Divyasree:~$ cd fifo_project
divya@Divyasree:~/fifo_project$ pwd
/home/divya/fifo_project
divya@Divyasree:~/fifo_project$ nano server.c
divya@Divyasree:~/fifo_project$ ^C
divya@Divyasree:~/fifo_project$ nano client.c
divya@Divyasree:~/fifo_project$ gcc server.c -o server
divya@Divyasree:~/fifo_project$ gcc client.c -o client
divya@Divyasree:~/fifo_project$ ls
client  client.c  server  server.c
divya@Divyasree:~/fifo_project$ ./server
Server started...
Waiting for client message...
Client sent: hello
Response sent: Hello Client! Message received.

Waiting for client message...
Client sent: How are you?
Response sent: Server processed: How are you?

Waiting for client message...
^C
divya@Divyasree:~/fifo_project$ |
```

```
GNU nano 7.2 server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define REQUEST_FIFO "request_fifo"
#define RESPONSE_FIFO "response_fifo"

int main() {
    char message[100];

    // Create FIFOs
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for client message...\n");

    while (1) {

        // Open request FIFO for reading
        int request_fd = open(REQUEST_FIFO, O_RDONLY);

        // Read message from client
        int n = read(request_fd, message, sizeof(message) - 1);
        close(request_fd);
```

```
GNU nano 7.2 Server.c
message[n] = '\0';

printf("Client sent: %s\n", message);

// Process the message
char response[200];

if (strcmp(message, "hello") == 0) {
    strcpy(response, "Hello Client! Message received.");
}
else if (strcmp(message, "exit") == 0) {
    strcpy(response, "Server shutting down.");
}
else {
    snprintf(response, sizeof(response),
             "Server processed: %s", message);
}

// Open response FIFO for writing
int response_fd = open(RESPONSE_FIFO, O_WRONLY);

write(response_fd, response, strlen(response) + 1);

close(response_fd);

printf("Response sent: %s\n\n", response);

if (strcmp(message, "exit") == 0) {
    break;
}
```

```
printf("Response sent: %s\n\n", response);

if (strcmp(message, "exit") == 0) {
    break;
}

printf("Waiting for client message...\n");
}

}

unlink(REQUEST_FIFO);
unlink(RESPONSE_FIFO);

printf("Server terminated.\n");

return 0;
}
```

```
GNU nano 7.2 client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define REQUEST_FIFO "request_fifo"
#define RESPONSE_FIFO "response_fifo"

int main() {
    char message[100];
    char response[200];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    // Remove newline
    message[strcspn(message, "\n")] = '\0';

    // Open request FIFO
    int request_fd = open(REQUEST_FIFO, O_WRONLY);

    // Send message to server
    write(request_fd, message, strlen(message) + 1);

    close(request_fd);

    // Open response FIFO
    int response_fd = open(RESPONSE_FIFO, O_RDONLY);
```

```
    // Read server response
    int n = read(response_fd, response, sizeof(response) - 1);

    close(response_fd);

    if (n > 0) {
        response[n] = '\0';
        printf("Server response: %s\n", response);
    }

    return 0;
}
```

```
divya@Divyasree:~$ cd fifo_project
divya@Divyasree:~/fifo_project$ ./client
Enter message: hello
Server response: Hello Client! Message received.
divya@Divyasree:~/fifo_project$ ./client
Enter message: How are you?
Server response: Server processed: How are you?
divya@Divyasree:~/fifo_project$ |
```

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
divya@Divyasree:~$ mkdir signal_project
divya@Divyasree:~$ cd signal_project
divya@Divyasree:~/signal_project$ pwd
/home/divya/signal_project
divya@Divyasree:~/signal_project$ nano signal_handler.c
divya@Divyasree:~/signal_project$ gcc signal_handler.c -o signal_handler
divya@Divyasree:~/signal_project$ ls
signal_handler  signal_handler.c
divya@Divyasree:~/signal_project$ ./signal_handler
Signal handling program started.
Process ID (PID): 761
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received! Interrupt signal handled.
Program is running...
Program is running...
^C
SIGINT received! Interrupt signal handled.
Program is running...
^C
SIGINT received! Interrupt signal handled.
Program is running...
Program is running...
Program is running...
kill -SIGINT 12345
Program is running...
Program is running...
Program is running...
```

```
Process ID (PID): 761
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received! Interrupt signal handled.
Program is running...
Program is running...
^C
SIGINT received! Interrupt signal handled.
Program is running...
^C
SIGINT received! Interrupt signal handled.
Program is running...
Program is running...
Program is running...
kill -SIGINT 12345
Program is running...
Program is running...
Program is running...
kill -SIGINT 12345
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
divya@Divyasree: ~/signal_pr X + v
divya@Divyasree:~$ cd ^C
divya@Divyasree:~$ cd signal_project
divya@Divyasree:~/signal_project$ kill -SIGINT 12345
-bash: kill: (12345) - No such process
divya@Divyasree:~/signal_project$ kill -SIGUSR1 12345
-bash: kill: (12345) - No such process
divya@Divyasree:~/signal_project$ kill -SIGTERM 12345
-bash: kill: (12345) - No such process
divya@Divyasree:~/signal_project$ kill -SIGTERM PID
-bash: kill: PID: arguments must be process or job IDs
divya@Divyasree:~/signal_project$ |
```

Code

```
GNU nano 7.2 signal_handler.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler function
void signal_handler(int signal) {
    if (signal == SIGINT) {
        printf("\nSIGINT received! Interrupt signal handled.\n");
    }
    else if (signal == SIGTERM) {
        printf("\nSIGTERM received! Termination signal handled.\n");
    }
    else if (signal == SIGUSR1) {
        printf("\nSIGUSR1 received! User-defined signal handled.\n");
    }
    fflush(stdout);
}

int main() {
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
```

```
int main() {
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");

    // Keep program running
    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```